Disaster Microservice Design (1 Service Only)

Student name: ___Pakin Maneerat____ Student id: __6609650558__

Disaster type: ☐ flood ☐ fire ☐ earthquake ☐ generic

---

1) Who are you helping? (pick 1)

☐ Citizen
☐ Dispatcher
☐ Rescue team
☐ Shelter staff
☐ Hospital staff
☐ Other: _____

2) What problem are you solving? (1 sentence)

Example: *"Many people report problems, but we don't know which one is urgent."*

Your problem: "The rescue team has limited resources and cannot distinguish which houses have vulnerable groups or which need assistance first."

3) Service name (verb + thing)

Examples:

- *HelpRequest Service*
- *Dispatch Service*
- *ShelterStatus Service*
- *EmergencyAlert Service*

Your service name: ____PriorityCase Service____

4) What is job of the service (i.e., purpose)? (1 sentence)

Example: *"This service stores help requests and lets dispatchers track status."*

Job/Purpose: Calculate the urgency score for each house by analyzing the water level along with the information of the residents in the house to prioritize the rescue efforts.

5) Data your service stores (6–10 fields)

Fill only what YOUR service owns.

Example fields: id, location, description, severity, status, timeReported

Your fields:

1. Request_id
2. location
3. water_level
4. description
5. priority_score
6. status
7. reported_time
8. _____ (optional)

6) 3 API actions (just write the idea)

(A) Create new record
Example: *"Create a new help request"*

POST: /create "Create a new request"

(B) View / Search
Example: *"Show all urgent requests in Zone A"*

GET: /queue "Show urgent queue"

(C) Update status
Example: *"Change status to DISPATCHED"*

PATCH: /queue/rq-001/update "Change status to Success"

7) Status values (choose 4–5)

Example: *NEW → VERIFIED → DISPATCHED → DONE*

Your status flow: New, Evaluating, On_mission, Done

8) One failure case (what breaks? and what should the service do?)

Example: *"If the service crashes, it should not lose data."*

Failure case: "The priority score calculation system is malfunctioning or running too slowly, causing data not to display on the screen."

Expected behavior: "The system must switch to basic mode, sorting cases by the time they were reported instead, so that rescuers still have access to information."

ภาพรวมของบริการ (Service Overview)
PriorityCase Service

1. Service Owner

นายภคิน มณีรัตน์ รหัสนักศึกษา 6609650558 ภาคพิเศษ

2. Service Purpose

PriorityCase Service เป็นบริการที่รับผิดชอบการรับคำร้องขอความช่วยเหลือ
และจัดลำดับผู้ที่ได้รับผลกระทบจากภัยพิบัติ
โดยทำหน้าที่เป็นตัวช่วยวิเคราะห์ว่าเคสไหนควรได้รับการช่วยเหลือก่อน
โดยจะคำนวนคะแนนความเร่งด่วนจากข้อมูลผู้อยู่อาศัย ประเภทภัยพิบัติ
และแสดงลำดับที่วิเคราะห์มาแล้วให้แก่ผู้เกี่ยวข้อง
เพื่อให้การช่วยเหลือผู้ได้รับผลกระทบมีประสิทธิภาพมากขึ้น

3. Pain Point ที่แก้ไข

ในสถานการณ์ภัยพิบัติ บริการนี้จะช่วยแก้ปัญหาการช่วยไม่ถูกจุด
โดยเปลี่ยนจากการทำงานตามลำดับเวลาที่แจ้งหรือการช่วยเหลือหน้าสถานการณ์
มาเป็นการทำงานตามความเร่งด่วนของผู้ได้รับผลกระทบ เพื่อลดความสูญเสียให้ได้มากที่สุด

3. Target Users

- เจ้าหน้าที่กู้ภัย
- ผู้แจ้งข้อมูลแก่เจ้าหน้าที่กู้ภัย
- ประชาชน

4. Service Boundary

- In-scope Responsibilities (สิ่งที่บริการนี้รับผิดชอบ)
    - รับและบันทึกข้อมูล
    - คำนวณคะแนนความเร่งด่วน
    - จัดลำดับคิวงาน
    - จัดการแผนสำรองเมื่อระบบขัดข้อง
    - บันทึกผลการปฏิบัติงาน

- Out-of-scope / Not Responsible For (ไม่รับผิดชอบ)
    - การมอบหมายงาน
    - ข้อมูลศูนย์พักพิง
    - การส่ง SMS หรือประกาศเตือน

5. Autonomy / Decision Logic
บริการมีความเป็นอิสระในการตัดสินใจเกี่ยวกับ:

- การกำหนดเกณฑ์การคำนวนความเร่งด่วน
- การจัดลำดับคิวงาน
- การสลับโหมดการทำงานเมื่อบริการทำงานผิดพลาด

การตัดสินใจอิงจาก:

- ข้อมูลผู้อยู่อาศัย
- แนวโน้มของสถานการณ์
- ระยะเวลาการรอคอย

บริการสามารถตัดสินใจได้เองภายใต้ business rules ที่กำหนด
<u>โดยไม่ต้องรอ/ต้องรอ</u>การอนุมัติจากมนุษย์ในกรณีปกติ

6. Owned Data

- Personal Data
  ข้อมูลผู้อยู่อาศัยที่ได้รับมาจากการส่งข้อมูล
  ควรอยู่ภายใต้บริการนี้เพราะเป็นข้อมูลที่นำมาใช้วิเคราะห์คำนวนคะแนนความเร่งด่วน
  ซึ่งบริการนี้เป็นผู้ใช้ข้อมูลโดยตรง
- Operational state
  ข้อมูลสถานะของ Request ที่ได้รับ
  ต้องอยู่ภายใต้บริการนี้เพื่อให้สามารถควบคุมความถูกต้องของลำดับคิวและเพื่ออัปเดตสถานะเมื่อได้ทำงานเสร็จเรียบร้อยแล้ว
- Priority Score
  ข้อมูลคะแนนความเร่งด่วนที่ระบบคำนวณได้และลำดับคิวในปัจจุบัน
  ข้อมูลนี้ต้องอยู่ภายใต้บริการนี้เพราะเป็นผลลัพธ์โดยตรงจาก Business Logic
  หลักของบริการ เพื่อใช้ในการจัดลำดับการเข้าช่วยเหลือให้แก่กู้ภัย

7. Linked Data (Reference Only)

- ข้อมูลสถานการณ์ในพื้นที่นั้นๆ เพื่อช่วยประกอบการคำนวนคะแนนความเร่งด่วน

8. Non-Functional Requirements

- ระบบต้องมีความพร้อมใช้งานสูง อย่างน้อย **99.9%** ในช่วงที่เกิดเหตุภัยพิบัติ
- การคำนวณคะแนนความเร่งด่วน ต้องเสร็จสิ้นภายใน **ไม่เกิน 1 นาที** หลังจากได้รับข้อมูล
- หากระบบคำนวณอัตโนมัติขัดข้อง ระบบต้องสามารถสลับไปทำงานในโหมดพื้นฐานได้ทันที
- ข้อมูลสถานะการช่วยเหลือ
  ต้องมีความถูกต้องแม่นยำและไม่สูญหายเมื่อระบบเกิดการขัดข้อง

Synchronous Function Contract

API Contract #1: Get Urgent Rescue Queue

ข้อมูลทั่วไป

- **Name:** Get Urgent Rescue Queue
- **Method:** GET
- **Path:** /v1/urgent
- **Type:** Synchronous

คำอธิบาย

ดึงรายการผู้ประสบภัยที่ยังไม่ได้รับการช่วยเหลือ

Request

Path/Query Params

- priority_score

Headers

- Accept: application/json

Body: ไม่มี

Response

Success: 200 OK


```
[
  {
    "request_id": "req_001",
    "location": "13.9989, 100.6177",
    "priority_score": 95.5,
    "status": "New",
   "water_level": 100,
    "reported_time": "2026-03-03T22:30:00Z",
```

"description": "บ้านชั้นเดียว มีผู้ป่วยติดเตียง 2 คน"

}

]

Error:

500 Internal Server Error: เมื่อระบบคำนวณขัดข้อง

401 Unauthorized

Dependency / Reliability

- ในกรณีที่ระบบคำนวณคะแนนอัตโนมัติล้มเหลว
  ระบบจะสลับไปดึงข้อมูลโดยเรียงลำดับตามเวลาที่ได้รับแจ้ง
  เพื่อให้เจ้าหน้าที่กู้ภัยยังคงเห็นข้อมูลพื้นฐานและปฏิบัติงานต่อไปได้โดยไม่หยุดชะงัก

API Contract #2: Create Rescue Request

ข้อมูลทั่วไป

- **Name:** Create Rescue Request
- **Method:** POST
- **Path:** /v1/newrequest
- **Type:** Synchronous

คำอธิบาย

ใช้สำหรับรับข้อมูลจากประชาชนเข้าสู่ระบบ
โดยบริการจะทำการบันทึกข้อมูลและคำนวณคะแนนความเร่งด่วนเบื้องต้นทันที
เพื่อนำเข้าสู่คิวงานของกู้ภัย

Request

Path/Query Param

- ไม่มี

Headers

- Content-Type: application/json

Body

{

```
  "location": "13.9989, 100.6177",
  "water_level": 100,
  "description": "มีกลุ่มเปราะบางในบ้าน 2 คน ต้องการความช่วยเหลือด่วน"
}
```

Response

Success: 201 Created

```
{
  "request_id": "req_002",
  "priority_score": 88.5,
  "status": "New",
  "reported_time": "2026-03-03T22:50:00Z"
}
```

Error: 400 Bad Request: ข้อมูลที่ส่งมาไม่ครบถ้วนหรือไม่ถูกต้องตามรูปแบบ

Dependency / Reliability

- ต้องเชื่อมต่อกับฐานข้อมูลเพื่อบันทึกข้อมูลใหม่
- หากส่วนคำนวณคะแนนขัดข้อง ระบบจะกำหนดคะแนนเริ่มต้น และบันทึกสถานะเป็น New เพื่อให้ข้อมูลเข้าสู่ระบบได้ก่อน

API Contract #3: Get Request by ID

ข้อมูลทั่วไป

- **Name:** Get Request Details by ID
- **Method:** GET
- **Path:** /v1/urgent/{request_id}
- **Type:** Synchronous

คำอธิบาย

ใช้สำหรับเรียกดูข้อมูลรายละเอียดทั้งหมดของเคสที่ระบุผ่าน request_id รวมถึงคะแนนความเร่งด่วนและสถานะล่าสุด เพื่อให้เจ้าหน้าที่กู้ภัยหรือ Dispatcher เห็นข้อมูลเชิงลึกก่อนเข้าปฏิบัติการ

Request

Path/Query Param

- request_id

Headers

- Accept: application/json

Body ไม่มี

Response

Success: 200 OK


```
{
  "request_id": "req_003",
  "location": "13.9989, 100.6177",
  "description": "มีกลุ่มเปราะบาง 2 คน ต้องการความช่วยเหลือด่วน",
  "priority_score": 88.5,
  "water_level": 150,
  "status": "Evaluating",
  "reported_time": "2026-03-03T22:50:00Z"
}
```


Error: 404 Not Found

401 Unauthorized

Dependency / Reliability

- ต้องเข้าถึงฐานข้อมูลหลักที่จัดเก็บ เพื่อดึงข้อมูลสถานะล่าสุด

API Contract #4: Update Mission Status

ข้อมูลทั่วไป

- **Name:** Update Mission Status
- **Method:** PATCH
- **Path:** /v1/urgent/{request_id}/status
- **Type:** Synchronous

คำอธิบาย

ใช้สำหรับให้ทีมกู้ภัยหรือ Dispatcher เปลี่ยนสถานะของเคสตามความคืบหน้าของงานจริง เพื่อให้ข้อมูลในระบบเป็นปัจจุบันและป้องกันการทำงานซ้ำซ้อน

Request

Path/Query Param

- request_id

Headers

- Accept: application/json

Body

```
{
  "status": "Success",
}
```

Response

Success: 200 OK

```
{
  "request_id": "req_003",
  "new_status": "Success",
  "updated_at": "2026-03-03T23:05:00Z"
}
```

Error: 404 Not Found

409 Conflict

Dependency / Reliability

- ต้องทำการตรวจสอบสถานะปัจจุบันในฐานข้อมูลก่อนแก้ไข
-

API Contract #5: Delete Rescue Request

ข้อมูลทั่วไป

- **Name:** Delete Rescue Request
- **Method:** DELETE
- **Path:** /v1/urgent/{request_id}
- **Type:** Synchronous

คำอธิบาย

ใช้สำหรับลบรายการแจ้งเหตุออกจากระบบในกรณีที่ตรวจสอบพบว่าเป็นข้อมูลซ้ำหรือเป็นการแจ้งเหตุเท็จ

Request

Path/Query Param

- request_id

Headers

- Accept: application/json

Body ไม่มี

Response

Success: 200 OK หรือ 204 No Content

```
{
  "message": "Request req_003 has been successfully deleted.",
  "deleted_at": "2026-03-03T23:45:00Z"
}
```

Error: 403 Forbidden

404 Not Found

Dependency / Reliability

- บริการต้องทำการลบข้อมูลออกจากฐานข้อมูลหลักของตนเอง

Asynchronous Function Contract

Message Contract #1: ArchiveLogDisasterToS3

ข้อมูลทั่วไป

- **Message Name:** DisasterRecordDeletedEvent
- **Interaction Style:** Event-Driven
- **Producer:** ผู้ใช้งาน
- **Consumer:** Lambda
- **Channel/Queue:** DynamoDB Streams
- **Version:** v1

คำอธิบาย

เมื่อมีการลบข้อมูล จากตารางหลัก ระบบจะเอาข้อมูลเดิม และทำการสำรองข้อมูลในรูปแบบ JSON File ลงใน Amazon S3

Request

Message Headers

EventSource: aws:dynamodb

EventName: REMOVE

Message Body


```
{
  "Records": [
    {
      "eventID": "unique_id",
      "eventName": "REMOVE",
      "dynamodb": {
        "OldImage": {
          "request_id": {"S": "req_001"},
          "incident_type": {"S": "ไฟไหม้"},
          "description": {"S": "ไฟไหม้หญ้า..."},
          "priority_score": {"N": "100"}
        }
      }
```

```
    }
  ]
}
```

Field Definition:

Field	Type	Required	Description
eventName	String	Yes	ประเภทเหตุการณ์ (Remove)
OldImage	Map	Yes	ข้อมูลทั้งหมดของ Record ก่อนที่จะถูกลบออก
request_id	String	Yes	ID อ้างอิงของเหตุการณ์
incident_type	String	Yes	ประเภทของภัยพิบัติ

Validation Rules

1. ประมวลผลเฉพาะ Record ที่มี eventName เท่ากับ REMOVE
2. OldImage ต้องมีข้อมูล request_id อยู่ภายใน

Response

Message Headers

Content-Type: application/json

Success Message Body

```
{
  "request_id": "req_001",
  "incident_type": "ไฟไหม้",
  "description": "...",
  "archived_at": "2026-03-22T21:29:53Z"
}
```

Reject/Error Message Body

```
{
  "error": "S3:AccessDenied",
  "message": "Lambda does not have permission to write to S3 bucket."
}
```

Field Definition:

Field	Type	Required	Description
request_id	String	Yes	ID ของข้อมูลที่ถูกสำรอง
archived_at	Timestamp	Yes	เวลาที่ทำการสำรองข้อมูลลง S3

Validation Rules

1. ชื่อไฟล์ใน S3 ต้องมีโครงสร้าง archived/{request_id}_{timestamp}.json
2. ข้อมูลใน JSON ต้องเป็นแบบ UTF-8 เพื่อรองรับภาษาไทย

Service Data

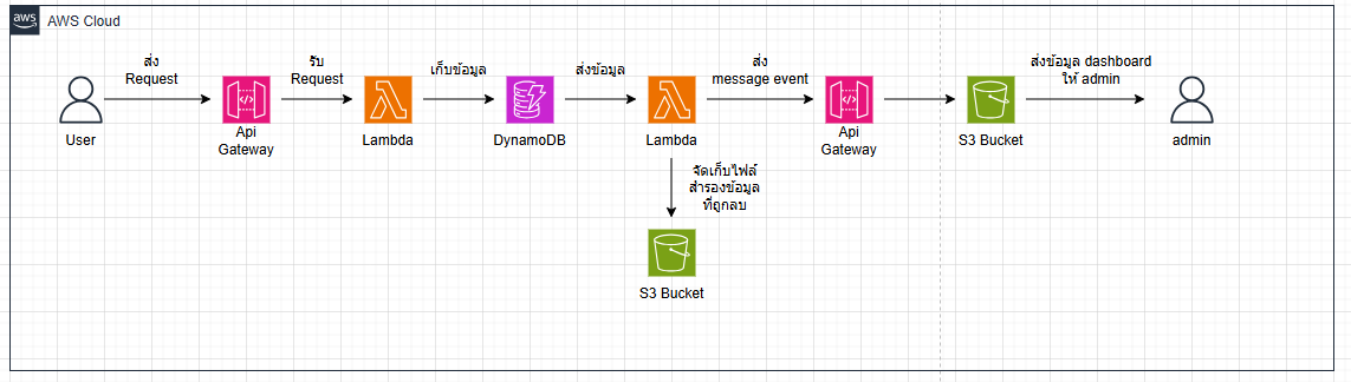
1) Request Data (Owned by this service)

Field Name	Type	Required	Description	Example
request_id	String	Yes	รหัสอ้างอิงเฉพาะของการแจ้งเหตุ	req_002
latitude	decimal	Yes	พิกัดละติจูด	13.987
longitude	decimal	Yes	พิกัดลองจิจูด	100.617
description	Text	Yes	รายละเอียดเพิ่มเติมเกี่ยวกับสถานการณ์	"บ้านชั้นเดียว มีคนชรา"
incident_type	String	Yes	ประเภทของภัยพิบัติ	Flood
reported_time	DateTime	Yes	วันและเวลาที่ได้รับแจ้งเหตุ	2026-03-03 22:30:00

2) Operational & Urgency State (Owned by this service)

Field Name	Type	Required	Description	Example
priority_score	decimal	Yes	คะแนนความด่วนที่ระบบคำนวณจากปัจจัยความเสี่ยงต่างๆ	92.5
status	String	Yes	สถานะปัจจุบันของเคส	New
last_updated	DateTime	Yes	วันเวลาล่าสุดที่มีการอัปเดตข้อมูลหรือสถานะในเคสนี้	2026-03-03 23:15:00

Service Architecture



Components

- **Amazon API Gateway:** ส่วนหน้าสำหรับรับ HTTP Requests จาก User และ Service ภายนอก
- **AWS Lambda (Synchronous):** ฟังก์ชันประมวลผลหลัก (Submit, Get, Update, Delete)
- **Amazon DynamoDB:** ฐานข้อมูลหลักสำหรับเก็บข้อมูลเหตุการณ์ภัยพิบัติ
- **AWS Lambda (Asynchronous):** ฟังก์ชันสำหรับสำรองข้อมูล (Archive) ที่ทำงานเบื้องหลัง
- **Amazon S3:** พื้นที่จัดเก็บไฟล์สำรอง

Explanation

Synchronous Workflow

เมื่อ User/Admin ส่งคำขอผ่าน API Gateway ระบบจะส่งต่อให้ Lambda (Sync) ทำการประมวลผลทันที เช่น Submit: รับข้อมูลภัยพิบัติ คำนวณ priority_score อัตโนมัติจากประเภทภัยพิบัติและคำสำคัญ (เช่น "ไฟไหม้", "ผู้สูงอายุ") แล้วบันทึกลง DynamoDB และตอบกลับสถานะการบันทึกสำเร็จทันที

Asynchronous Workflow

ระบบใช้สถาปัตยกรรมแบบ Event-Driven เพื่อแยกส่วนการสำรองข้อมูลออกจากระบบหลัก

- Trigger : เมื่อมีการลบข้อมูลใน DynamoDB ข้อมูลเดิมจะถูกส่งเข้าสู่ DynamoDB Streams
- Processing : Lambda (Async) จะถูก Triggered โดย Streams ให้ประมวลผลข้อมูลที่ถูกลบนั้น
- Archiving : ข้อมูลจะถูกแปลงเป็นไฟล์ JSON และจัดเก็บลงใน Amazon S3 ทันที กระบวนการนี้เกิดขึ้นแบบคู่ขนานโดยที่ Admin ไม่ต้องรอการบันทึกลง S3 เสร็จสิ้นก่อน ทำให้การใช้งานหน้าเว็บลื่นไหลและรวดเร็ว

Service Interaction

Upstream Service

- **IncidentReporter Service**
    - Interaction Style : Synchronous
    - รับการรายงานเหตุการณ์จากพลเมืองหรือเจ้าหน้าที่ในพื้นที่
      เพื่อให้ระบบทำการประมวลผลในขั้นตอนถัดไป

Downstream Service

- **VolunteerMatch Service**
    - Interaction Style : Synchronous
    - หลังจากที่ระบบของเราทำการบันทึกเหตุการณ์และคำนวนความเร่งด่วนเบื้องต้นแล้ว
      ระบบจะทำการส่งข้อมูลเหตุการณ์ที่มีความด่วนสูง ไปยังระบบปลายทาง
      เพื่อให้ระบบดังกล่าวทำการจับคู่กับหน่วยงานกู้ภัย

Link Github

https://github.com/pakinz007x/microservice_0558.gi